Gradient Descent for Least Squares

David Puelz
The University of Austin

1 The problem

Simple linear regression: $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$, for $i = 1, \ldots, n$. We fit (β_0, β_1) by minimizing the mean squared error,

$$L(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2 .$$

It helps to name the residual at the current guess: $e_i = y_i - \beta_0 - \beta_1 x_i$. Then $L = \frac{1}{n} \sum_i e_i^2$.

L is a bowl over the (β_0, β_1) plane: a quadratic in both parameters, with a single lowest point. That lowest point is the OLS fit.

2 The gradient

The gradient is the vector of partial derivatives. By the chain rule, $\frac{\partial}{\partial \beta} e_i^2 = 2 e_i \frac{\partial e_i}{\partial \beta}$, with $\frac{\partial e_i}{\partial \beta_0} = -1$ and $\frac{\partial e_i}{\partial \beta_1} = -x_i$. So

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^{n} e_i, \qquad \frac{\partial L}{\partial \beta_1} = -\frac{2}{n} \sum_{i=1}^{n} e_i \, x_i.$$

Read them in words:

- $\partial L / \partial \beta_0$ is -2 times the **average residual**. If the line sits too low on average ($e_i > 0$), the derivative is negative, so increasing β_0 lowers the loss.
- $\partial L / \partial \beta_1$ is -2 times the **average of residual times** x. If the misses line up with x, the slope is wrong and this term says which way to tilt it.

3 Setting the gradient to zero gives OLS

At the minimum both partials are zero:

$$\sum_i e_i = 0, \qquad \sum_i e_i \, x_i = 0.$$

These are the **normal equations**. The first gives $\bar{y} = \beta_0 + \beta_1 \bar{x}$, so $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$. Substituting into the second and simplifying,

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2},$$

the familiar OLS slope. So gradient descent, if it converges, lands exactly on `lm()`'s answer.

The two equations also say the residuals average to zero and are uncorrelated with x: the two facts you already know about OLS residuals.

4 The gradient descent update

Start anywhere, say $(\beta_0, \beta_1) = (0, 0)$, and repeatedly step *downhill*, opposite the gradient, with step size (learning rate) η:

$$\beta_0 \leftarrow \beta_0 + \eta \cdot \frac{2}{n} \sum_i e_i, \qquad \beta_1 \leftarrow \beta_1 + \eta \cdot \frac{2}{n} \sum_i e_i \, x_i,$$

recomputing the residuals e_i after every step. Each update nudges the line toward the points it is currently missing, in proportion to how much it misses them.

5 Matrix form

Stack the data: X is $n \times 2$ with rows $(1, x_i)$, and $\beta = (\beta_0, \beta_1)^\top$. Then

$$L(\beta) = \frac{1}{n} \|y - X\beta\|^2, \qquad \nabla L(\beta) = -\frac{2}{n} X^\top (y - X\beta).$$

Setting $\nabla L = 0$ gives $X^\top X \beta = X^\top y$, so $\hat{\beta} = (X^\top X)^{-1} X^\top y$. The update is

$$\beta \leftarrow \beta + \frac{2\eta}{n} X^\top (y - X\beta).$$

Nothing here depends on there being one predictor: the same two lines handle p predictors.

6 How big a step?

Let $H = \frac{1}{n} X^\top X$. Since $y - X\beta = (y - X\hat{\beta}) - X(\beta - \hat{\beta})$ and $X^\top (y - X\hat{\beta}) = 0$, one step changes the error $\beta - \hat{\beta}$ by a fixed matrix:

$$\beta_{k+1} - \hat{\beta} = (I - 2\eta H) \left(\beta_k - \hat{\beta} \right).$$

The error shrinks every step if and only if $|1 - 2\eta\lambda| < 1$ for every eigenvalue λ of H, that is,

$$0 < \eta < \frac{1}{\lambda_{\max}(H)}.$$

- Too small an η: $1 - 2\eta\lambda \approx 1$, and the error barely shrinks.
- Too large: $1 - 2\eta\lambda < -1$, so the error flips sign and grows each step. The fit overshoots and diverges.
- With centered, standardized x, $H \approx I$, so any $\eta < 1$ works. This is one reason to standardize predictors.

The one-parameter example on the slides, $L(\theta) = (\theta - 3)^2$, has $H = 1$: learning rate 0.4 converges and 1.05 diverges, exactly as this bound predicts.

7 *Minibatches*

Replace the average over all n cases with the average over a random minibatch B:

$$\nabla L_B(\beta) = -\frac{2}{|B|} X_B^\top (y_B - X_B\beta).$$

Because B is a random sample, ∇L_B is an **unbiased** estimate of the full gradient: right on average, noisy step to step. Each step is much cheaper, so minibatches usually reach the bottom in fewer passes through the data. **Stochastic gradient descent** is the special case of batch size 1.

8 *Why bother, when OLS has a formula?*

For linear regression you would not, since `lm()` solves the normal equations directly. Gradient descent earns its keep when

- n or p is so large that forming and solving $X^\top X\beta = X^\top y$ is impractical, or
- there is *no* closed form at all: logistic regression, and every neural network.

The recipe never changes: write the loss, take its gradient with the chain rule, and step downhill. For a neural network, backpropagation is just the chain rule organized to compute that gradient efficiently.

Code: the course script `gradient_descent_regression.R` runs this on simulated data, with full-batch, minibatch, and SGD paths plotted on the loss surface.